SREE NARAYANA GURU COLLEGE OF COMMERCE (AUTONOMOUS)

TYBSCIT ANDROID PRACTICAL

2026-27

Student Name: _____

Seat No: _____

INDEX

Module I: Android Fundamentals & UI Design

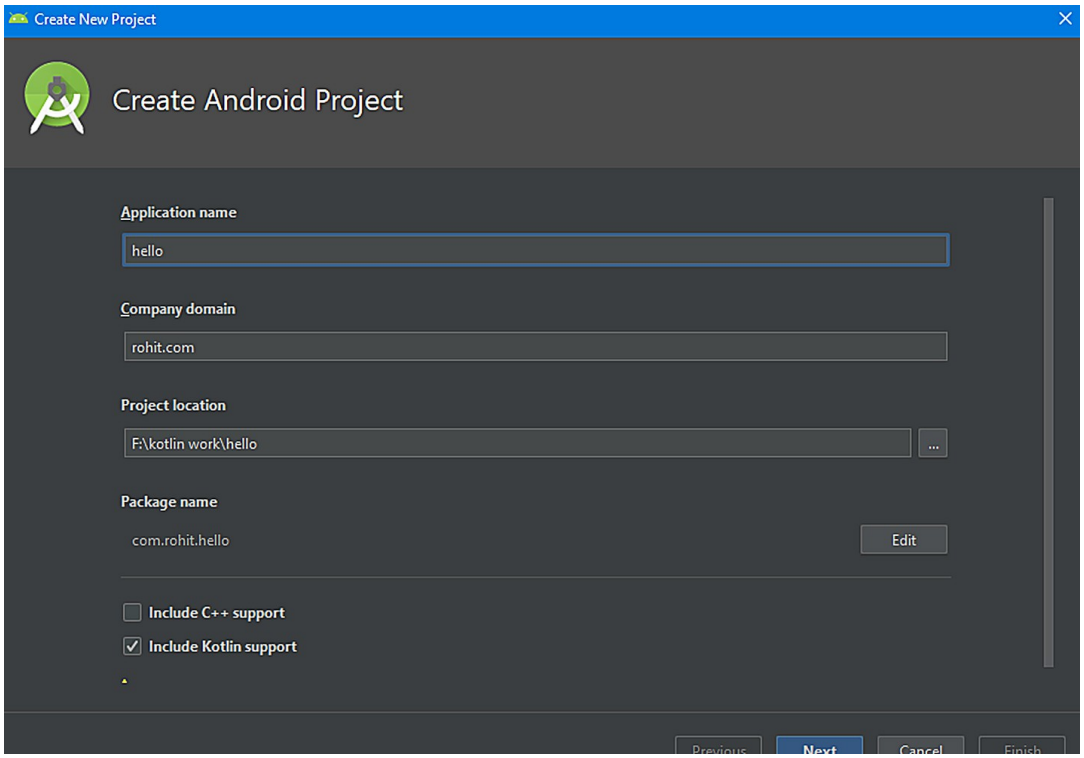
Sr. No	Title	Date	Signature
Android Basics and Resources			
1	Develop a simple “Hello World” Android application demonstrating project creation, Android Studio interface, Android components, AVD creation, and application execution.		
2	Develop an Android application demonstrating the use of Programming Resources (colors.xml, themes.xml, strings.xml, dimens.xml, drawable resources) and apply them in UI components.		
3	Develop an Android application demonstrating complete Activity Lifecycle methods and analyze execution using Logcat.		
Fragments and Activity Interaction			
4	Develop an Android application to create and integrate a Fragment and demonstrate Fragment lifecycle methods.		
5	Develop an Android application implementing Fragment-based UI navigation between multiple screens.		
6	Develop an Android application demonstrating communication between Activity and Fragment.		
Basic Layout Management			
7	Develop an Android application using LinearLayout (Horizontal and Vertical) to design structured UI.		
8	Develop an Android application using RelativeLayout to align UI components dynamically.		
9	Develop an Android application using TableLayout to design a structured Login/Registration Form.		
Android Layout and View Handling			
10	Develop an Android application using ScrollView to display long scrollable content.		
11	Develop an Android application using AbsoluteLayout demonstrating fixed positioning.		
12	Develop an Android application using FrameLayout to overlay UI elements.		
Adapter Based Views			
13	Develop an Android application using ListView with ArrayAdapter and implement item click handling.		
14	Develop an Android application using GridView with Custom Adapter and image selection handling.		
15	Develop an Android application implementing RecyclerView as a		

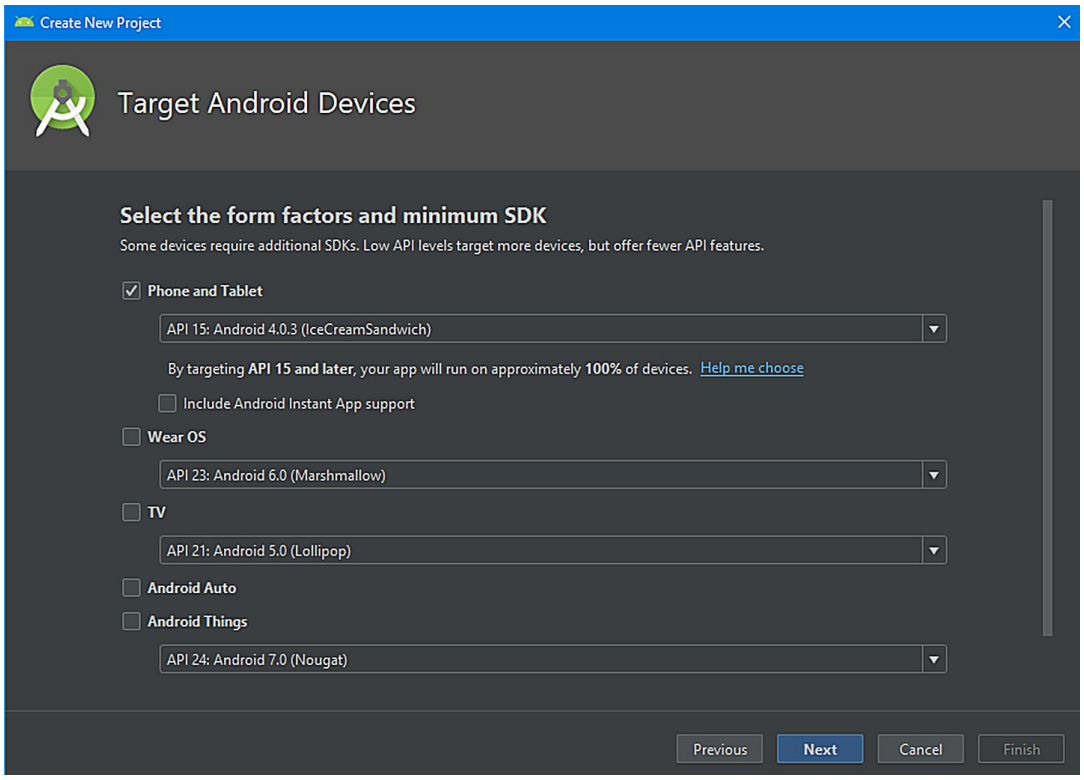
Sr. No	Title	Date	Signature
	modern alternative to ListView.		

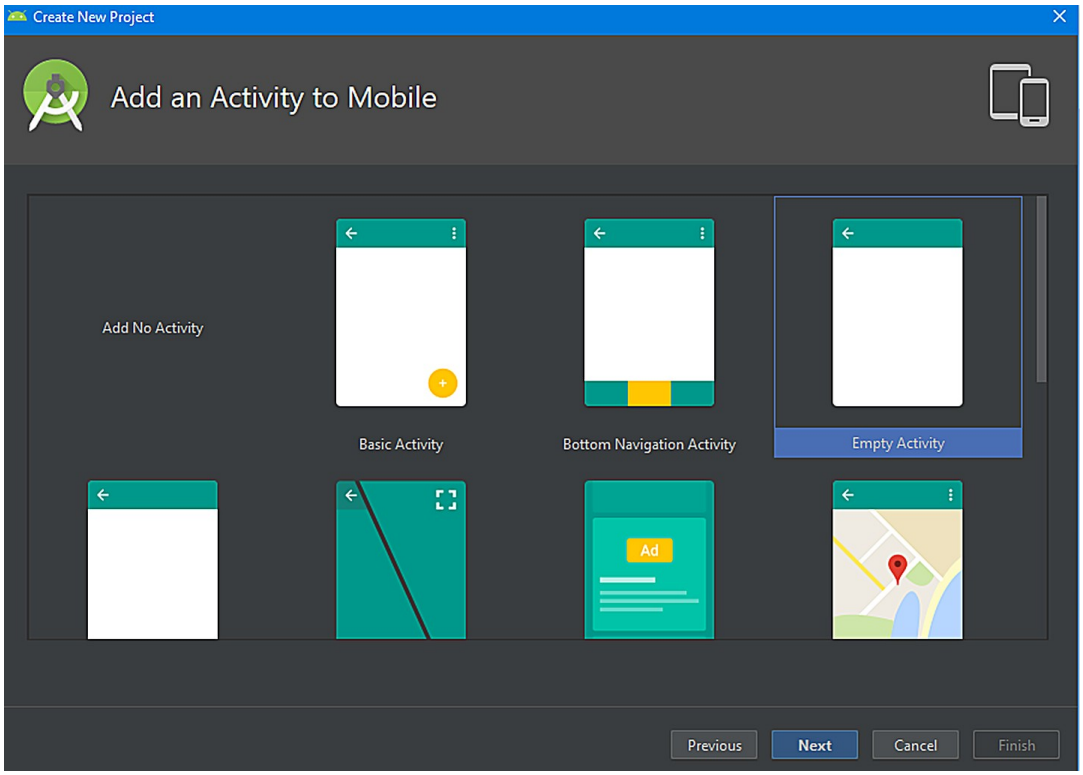
PRACTICAL NO. 1

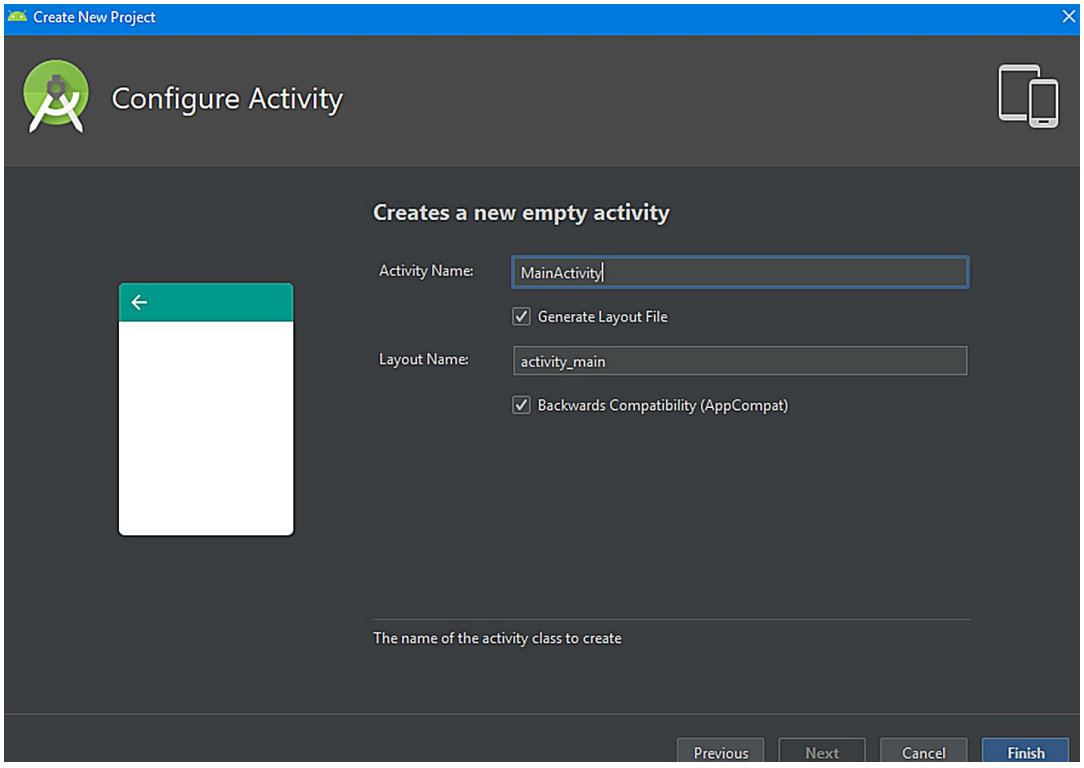
Develop a simple “Hello World” Android application demonstrating project creation, Android Studio interface, Android components, AVD creation, and application execution.

Creating a project:









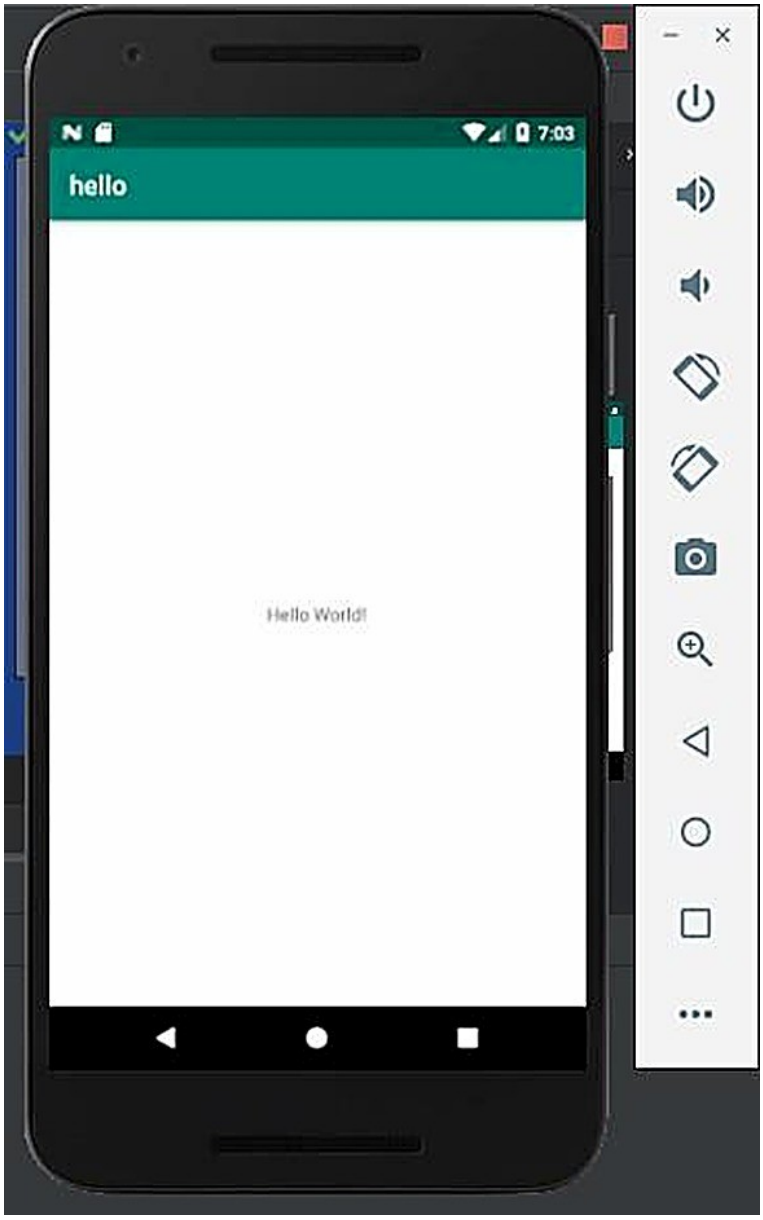
Activity_Main.kt

```
package com.rohit.hello
import android.support.v7.app.AppCompatActivity
import android.os.Bundle
class MainActivity : AppCompatActivity() {
override fun onCreate(savedInstanceState: Bundle?) {
super.onCreate(savedInstanceState)
setContentView(R.layout.activity_main)
}
}
```

activity_Main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<android.support.constraint.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:tools="http://schemas.android.com/tools"
xmlns:app="http://schemas.android.com/apk/res-auto"
android:layout_width="match_parent"
android:layout_height="match_parent"
tools:context=".MainActivity">
<TextView
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="Hello World!"
app:layout_constraintBottom_toBottomOf="parent"
app:layout_constraintLeft_toLeftOf="parent"
app:layout_constraintRight_toRightOf="parent"
app:layout_constraintTop_toTopOf="parent"/>
</android.support.constraint.ConstraintLayout>
```

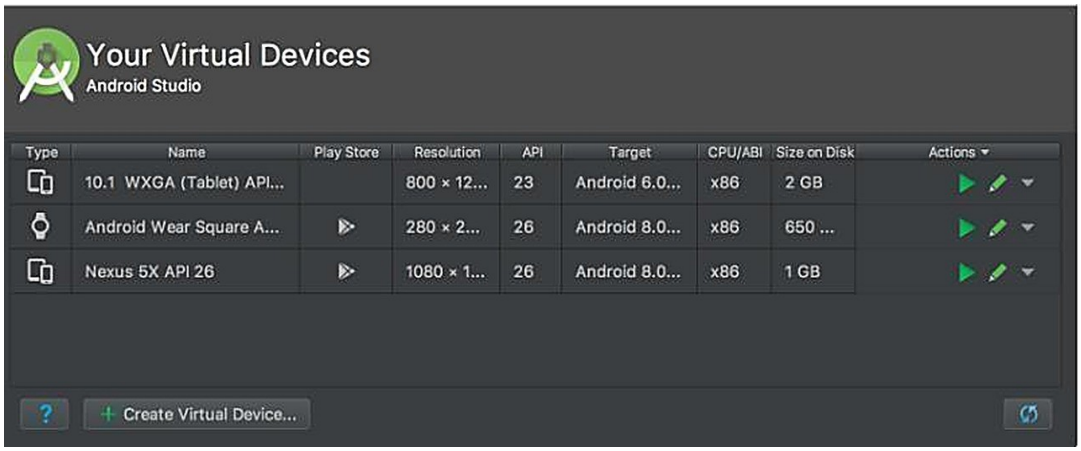
Application in AVD:

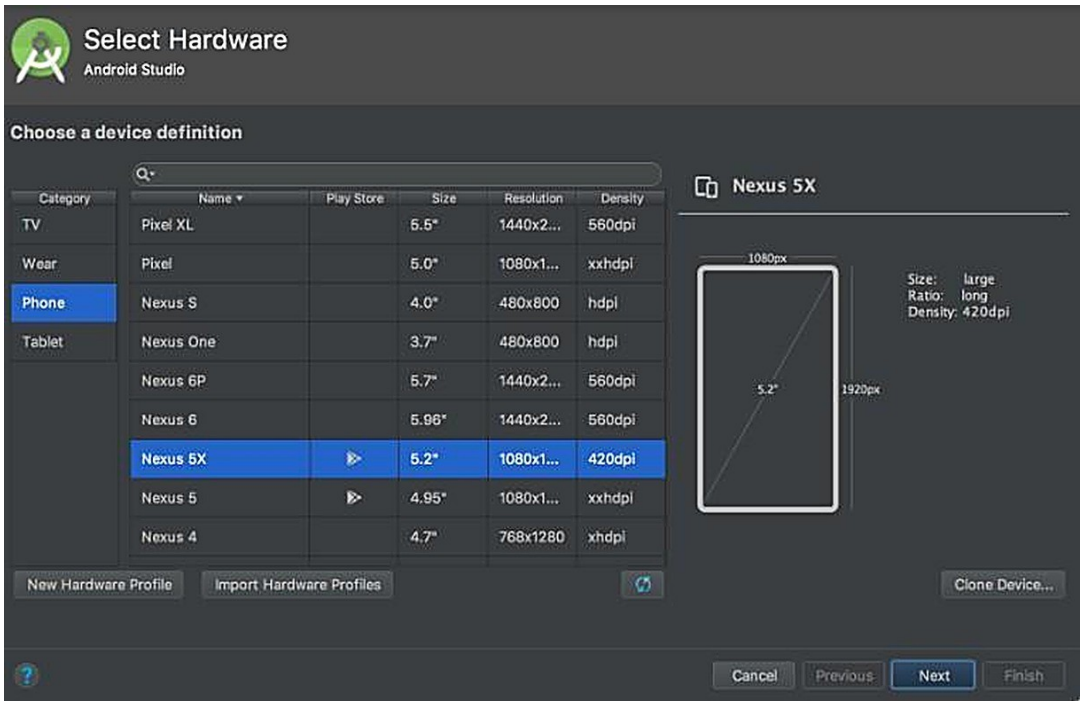


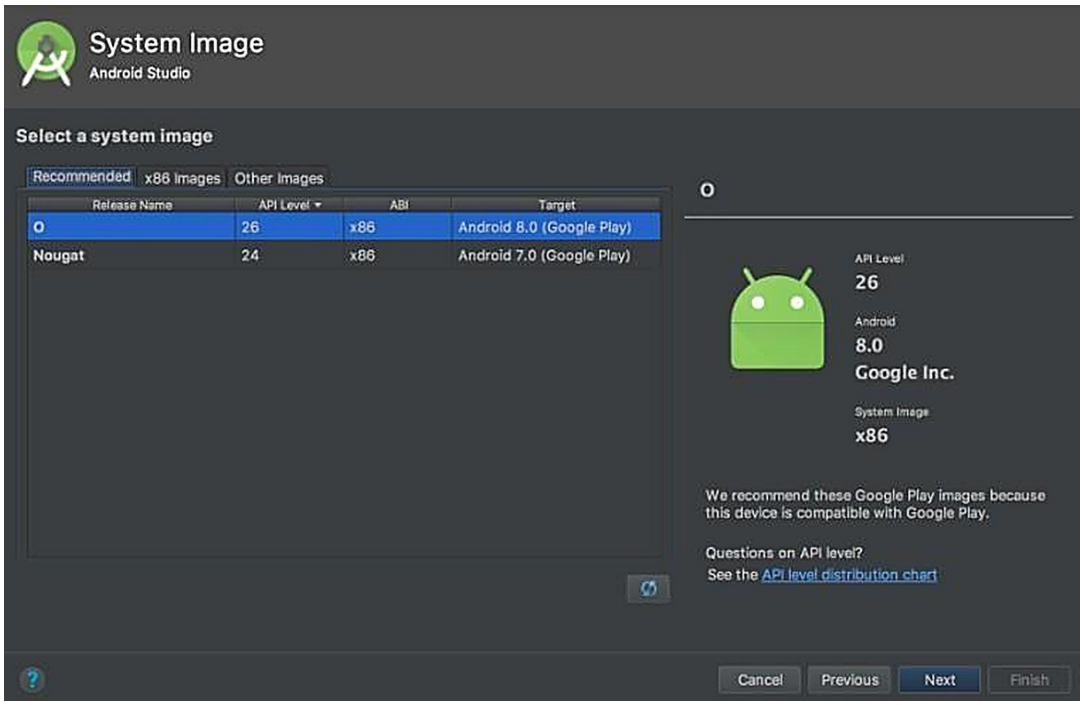
Create and manage virtual devices:

To open the AVD Manager, do one of the following:

- Select Tools > AVD Manager.
- Click AVD Manager AVD Manager icon in the toolbar.






System Image
Android Studio
Select a system image
Recommended
x86 Images
Other Images
Release Name
API Level
ABI
Target
O
26
x86
Android 8.0 (Google Play)
Nougat
24
x86
Android 7.0 (Google Play)
O
API Level
26
Android
8.0
Google Inc.
System Image
x86
We recommend these Google Play images because this device is compatible with Google Play.
Questions on API level?
See the API level distribution chart
Cancel
Previous
Next
Finish


12:49
April 3
Maps


Android Virtual Device (AVD)
Android Studio
Verify Configuration
AVD Name
Nexus 5X API 26 2
Nexus 5X
5.2 1080x1920 xxhdpi
Change...
O
Android 8.0 x86
Change...
Startup orientation
Portrait
Landscape
Emulated Performance
Graphics:
Automatic
Device Frame
Enable Device Frame
Show Advanced Settings
AVD Name
The name of this AVD.
Cancel
Previous
Next
Finish

PRACTICAL NO. 2

Develop an Android application demonstrating the use of Programming Resources (colors.xml, themes.xml, strings.xml, dimens.xml, drawable resources) and apply them in UI components.

In this practical we will learn how to organize application resources, specify alternative resources and access them in your applications.

Types of Resources

The following are the most common types of resources within Android apps:

Name	Folder	Description
Drawables	drawable	Bitmap files or XML files that act as graphics
Layout	layout	XML files that define a user interface layout
Menu	menu	XML files that define menus or action bar items
Values	values	XML files with values such as strings, integers, and colors.

In addition, note the following key files stored within the values folder mentioned above:

Name	File	Description
Colors	`res/values/colors.xml`	For color definitions such as text color
Strings	`res/values/strings.xml`	For string values such as the text for a title
Styles	`res/values/styles.xml`	For style values such as color of the AppBar

Example

Demonstration the use of various resource files

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout
xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:tools="http://schemas.android.com/tools"
xmlns:app="http://schemas.android.com/apk/res-auto"
android:layout_width="match_parent"
tools:context=".MainActivity"
android:background="@drawable/sky">
<ImageView
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:src="@drawable/sun"
android:id="@+id/img"
android:layout_centerHorizontal="true"
android:layout_alignParentTop="true"
android:layout_marginTop="117dp" />
<Button
android:id="@+id/btn"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="@string/button_name"
```

```
android:layout_below="@+id/img"
android:layout_centerHorizontal="true"
android:background="@color/btn_color"
android:layout_marginTop="123dp"
android:onClick="show"/>
</RelativeLayout>
```

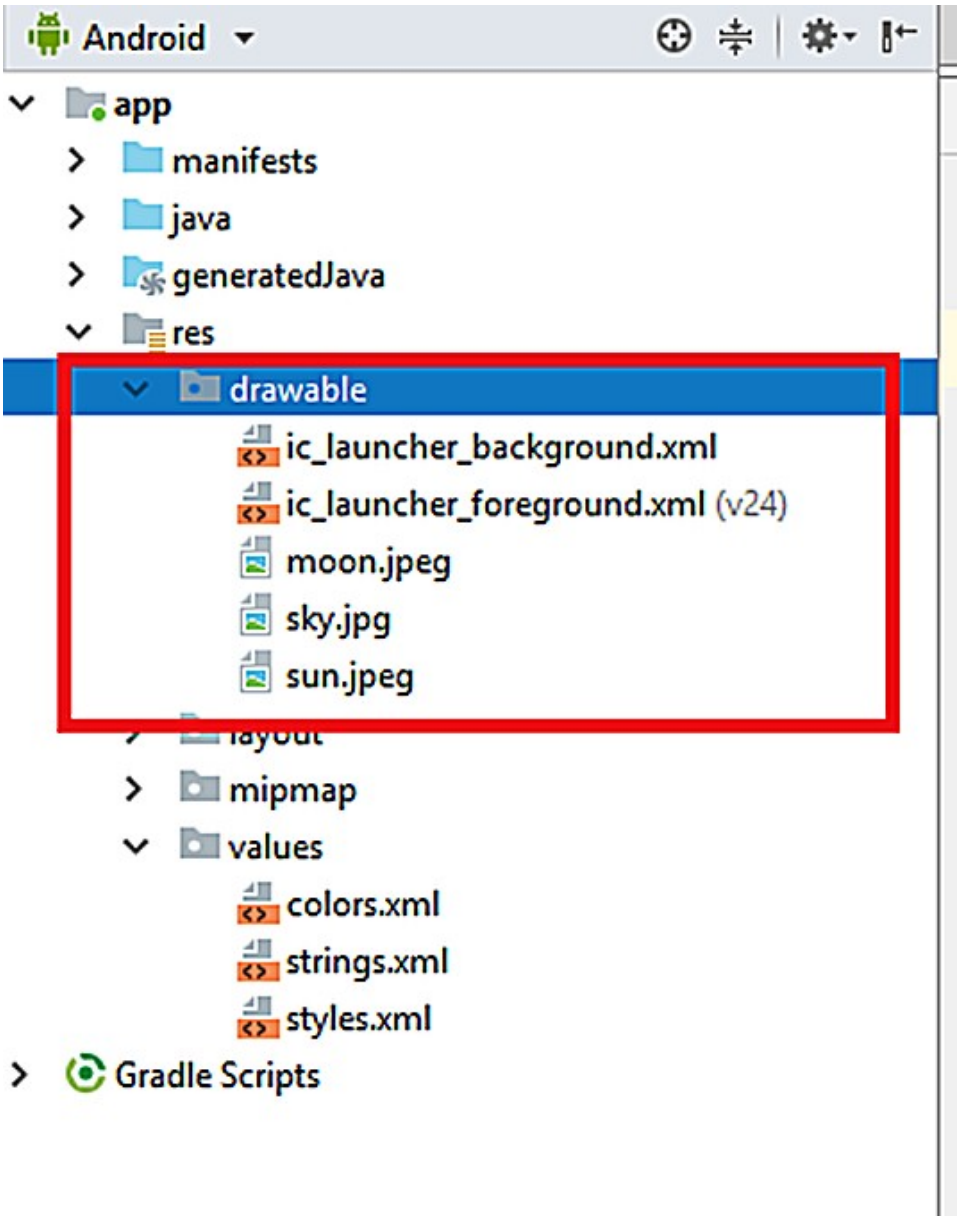
colors.xml

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
<color name="colorPrimary">#008577</color>
<color name="colorPrimaryDark">#00574B</color>
<color name="colorAccent">#D81B60</color>
<color name="btn_color">#FFC49662</color>
</resources>
```

string.xml

```
<resources>
<string name="app_name">Resources</string>
<string name="button_name">Click on me</string>
</resources>
```

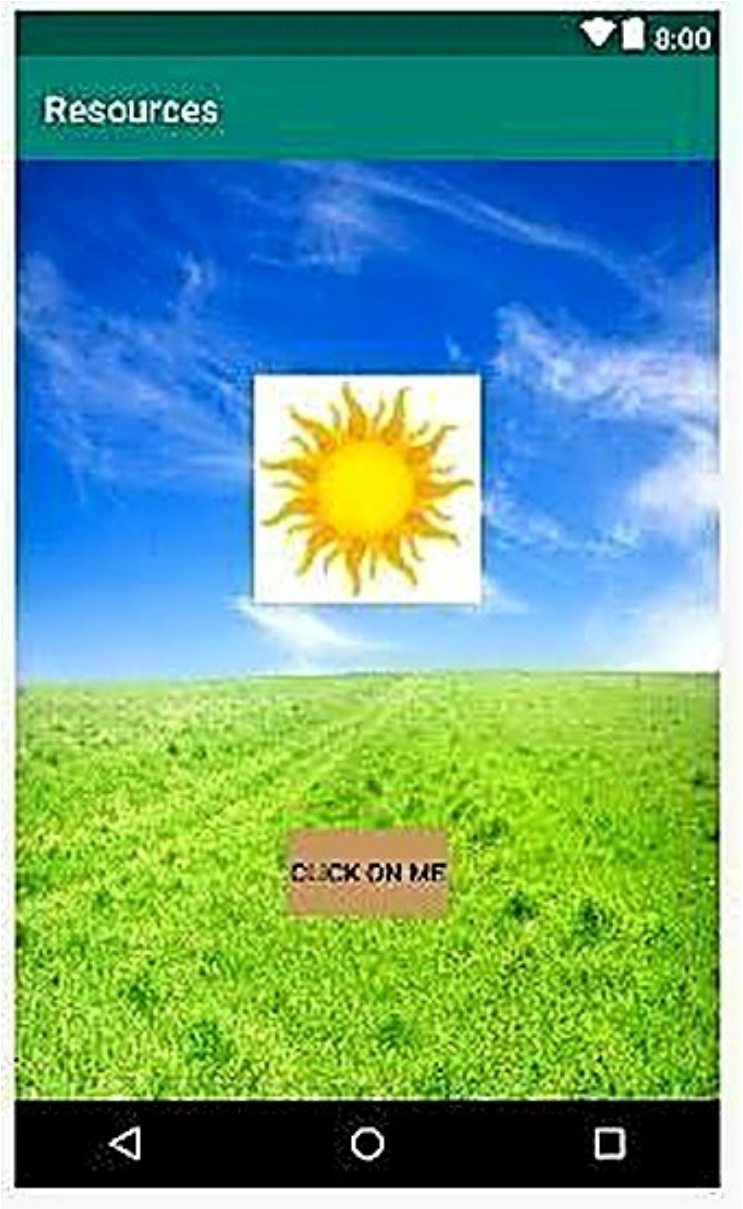
In Drawable add the image which you want to display



Mainactivity.xml

```
package com.example.admin.resources
import android.support.v7.app.AppCompatActivity
import android.os.Bundle
import android.view.View
import kotlinx.android.synthetic.main.activity_main.*
class MainActivity : AppCompatActivity() {
override fun onCreate(savedInstanceState: Bundle?) {
super.onCreate(savedInstanceState)
setContentView(R.layout.activity_main)
}
fun show(view: View){
img.setImageResource(R.drawable.moon)
}
}
```

Output:


8:00
Resources
CLICK ON ME

PRACTICAL NO. 3

Develop an Android application demonstrating complete Activity Lifecycle methods and analyze execution using Logcat.

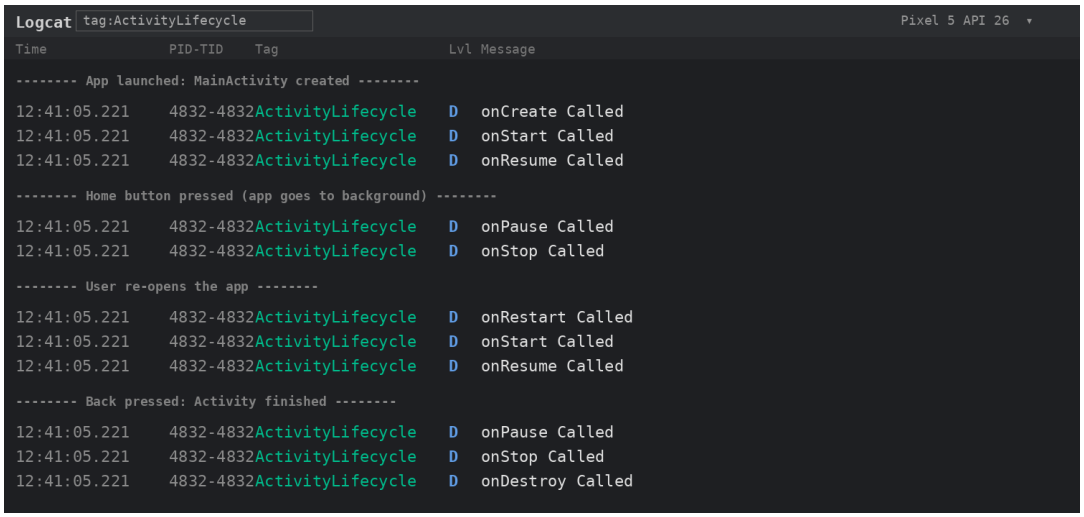
MainActivity.kt

```
class MainActivity : AppCompatActivity() {
private val TAG = "ActivityLifecycle"
override fun onCreate(savedInstanceState: Bundle?) {
super.onCreate(savedInstanceState)
enableEdgeToEdge()
setContentView(R.layout.activity_main)
Log.d(TAG, "onCreate Called")
}
override fun onStart() {
super.onStart()
Log.d(TAG, "onStart Called")
}
override fun onResume() {
super.onResume()
Log.d(TAG, "onResume Called")
}
override fun onPause() {
super.onPause()
Log.d(TAG, "onPause Called")
}
override fun onStop() {
super.onStop()
Log.d(TAG, "onStop Called")
}
override fun onRestart() {
super.onRestart()
Log.d(TAG, "onRestart Called")
}
override fun onDestroy() {
super.onDestroy()
Log.d(TAG, "onDestroy Called")
}
}
```

How to see the output in Logcat

1. Run your app.
2. Open View → Tool Windows → Logcat.
3. Select your device/emulator.
4. Select your app's process.
5. In the search box, type:
6. ActivityLifecycle

Output:


Logcat
tag:ActivityLifecycle
Pixel 5 API 26
Time PID-TID Tag Lvl Message
-------- App launched: MainActivity created --------
12:41:05.221 4832-4832 ActivityLifecycle D onCreate Called
12:41:05.221 4832-4832 ActivityLifecycle D onStart Called
12:41:05.221 4832-4832 ActivityLifecycle D onResume Called
-------- Home button pressed (app goes to background) --------
12:41:05.221 4832-4832 ActivityLifecycle D onPause Called
12:41:05.221 4832-4832 ActivityLifecycle D onStop Called
-------- User re-opens the app --------
12:41:05.221 4832-4832 ActivityLifecycle D onRestart Called
12:41:05.221 4832-4832 ActivityLifecycle D onStart Called
12:41:05.221 4832-4832 ActivityLifecycle D onResume Called
-------- Back pressed: Activity finished --------
12:41:05.221 4832-4832 ActivityLifecycle D onPause Called
12:41:05.221 4832-4832 ActivityLifecycle D onStop Called
12:41:05.221 4832-4832 ActivityLifecycle D onDestroy Called

PRACTICAL NO. 4

Develop an Android application to create and integrate a Fragment and demonstrate Fragment lifecycle methods.

Fragment Lifecycle

To do this Practical – 4 files

Activitymain.xml, Fragment.xml, MainActivity.kt, MyFragment.kt

Steps

Create a new project. After creating the new project create fragment: Right click on package (kotlin+java) → create fragment → Blank Fragment → Name give: MyFragment.

After creating Fragment you can see MyFragment.kt file and fragment.xml file.

Note: MainActivity.kt file and Activitymain.xml file already created in Android.

Now start doing the Practical

1) Activitymain.xml

```
<?xml version="1.0" encoding="utf-8"?>
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
android:id="@+id/fragmentContainer"
android:layout_width="match_parent"
android:layout_height="match_parent">
</FrameLayout>
```

Fragment.xml

```
<?xml version="1.0" encoding="utf-8"?>
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:tools="http://schemas.android.com/tools"
android:id="@+id/fragmentContainer"
android:layout_width="match_parent"
android:layout_height="match_parent"
tools:context=".MyFragment">
<!-- TODO: Update blank fragment layout -->
<TextView
android:layout_width="match_parent"
android:layout_height="match_parent"
android:text="@string/hello_blank_fragment" />
</FrameLayout>
```

MainActivity.kt

```
package com.example.fragmentprac
import android.os.Bundle
import androidx.activity.enableEdgeToEdge
import androidx.appcompat.app.AppCompatActivity
import androidx.core.view.ViewCompat
import androidx.core.view.WindowInsetsCompat
class MainActivity : AppCompatActivity() {
override fun onCreate(savedInstanceState: Bundle?) {
super.onCreate(savedInstanceState)
enableEdgeToEdge()
setContentView(R.layout.activity_main)
supportFragmentManager.beginTransaction()
```

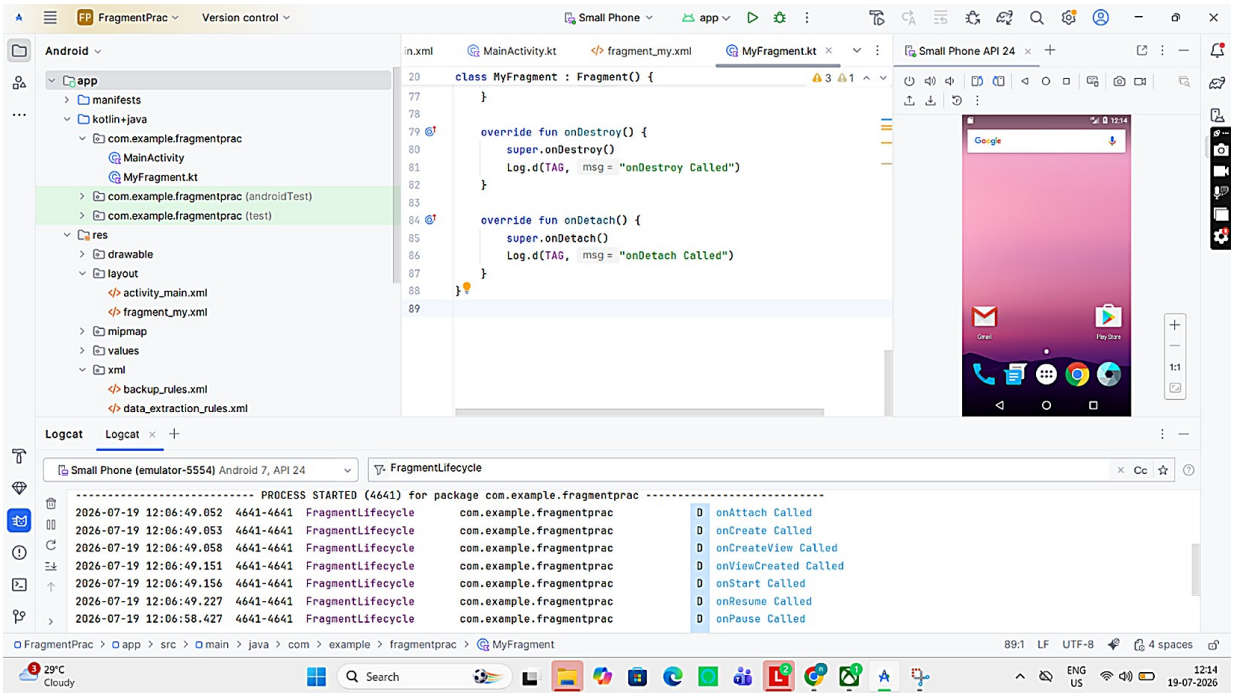
```
.replace(
R.id.fragmentContainer,
MyFragment()
)
.commit()
}
}
```

MyFragment.kt

```
package com.example.fragmentprac
import android.os.Bundle
import android.util.Log
import androidx.fragment.app.Fragment
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
// TODO: Rename parameter arguments, choose names that match
// the fragment initialization parameters, e.g. ARG_ITEM_NUMBER
private const val ARG_PARAM1 = "param1"
private const val ARG_PARAM2 = "param2"
/**
* A simple [Fragment] subclass.
* Use the [MyFragment.newInstance] factory method to
* create an instance of this fragment.
*/
class MyFragment : Fragment() {
private val TAG = "FragmentLifecycle"
override fun onAttach(context: android.content.Context) {
super.onAttach(context)
Log.d(TAG, "onAttach Called")
}
override fun onCreate(savedInstanceState: Bundle?) {
super.onCreate(savedInstanceState)
Log.d(TAG, "onCreate Called")
}
override fun onCreateView(
inflater: LayoutInflater,
container: ViewGroup?,
savedInstanceState: Bundle?
): View? {
Log.d(TAG, "onCreateView Called")
return inflater.inflate(
R.layout.fragment_my,
container,
false
)
}
override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
super.onViewCreated(view, savedInstanceState)
Log.d(TAG, "onViewCreated Called")
}
override fun onStart() {
super.onStart()
```

```
Log.d(TAG, "onStart Called")
}
override fun onResume() {
super.onResume()
Log.d(TAG, "onResume Called")
}
override fun onPause() {
super.onPause()
Log.d(TAG, "onPause Called")
}
override fun onStop() {
super.onStop()
Log.d(TAG, "onStop Called")
}
override fun onDestroyView() {
super.onDestroyView()
Log.d(TAG, "onDestroyView Called")
}
override fun onDestroy() {
super.onDestroy()
Log.d(TAG, "onDestroy Called")
}
override fun onDetach() {
super.onDetach()
Log.d(TAG, "onDetach Called")
}
}
```

Output



PRACTICAL NO. 5

Develop an Android application implementing Fragment-based UI navigation between multiple screens.

Application: Fragment-Based Navigation

Screens: 1. Home Fragment 2. Profile Fragment 3. Settings Fragment

```
└── app/kotlin/com.example.fragmentnavigation
├── MainActivity.kt
├── HomeFragment.kt
├── ProfileFragment.kt
└── SettingsFragment.kt
└── res
├── layout
│   ├── activity_main.xml
│   ├── fragment_home.xml
│   ├── fragment_profile.xml
│   └── fragment_settings.xml
└── values
└── strings.xml
```

activitymain.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:orientation="vertical">
<FrameLayout
android:id="@+id/fragmentContainer"
android:layout_width="match_parent"
android:layout_height="0dp"
android:layout_weight="1" />
<LinearLayout
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:orientation="horizontal">
<Button
android:id="@+id/btnHome"
android:layout_width="0dp"
android:layout_height="wrap_content"
android:layout_weight="1"
android:text="Home" />
<Button
android:id="@+id/btnProfile"
android:layout_width="0dp"
android:layout_height="wrap_content"
android:layout_weight="1"
android:text="Profile" />
<Button
android:id="@+id/btnSettings"
android:layout_width="0dp"
android:layout_height="wrap_content"
```

```
android:layout_weight="1"
android:text="Settings" />
</LinearLayout>
</LinearLayout>
```

MainActivity.kt

```
package com.example.fragmentnavigation
import android.os.Bundle
import androidx.appcompat.app.AppCompatActivity
import androidx.fragment.app.Fragment
class MainActivity : AppCompatActivity() {
override fun onCreate(savedInstanceState: Bundle?) {
super.onCreate(savedInstanceState)
setContentView(R.layout.activity_main)
// Display Home Fragment initially
loadFragment(HomeFragment())
findViewById<android.widget.Button>(R.id.btnHome).setOnClickListener {
loadFragment(HomeFragment())
}
findViewById<android.widget.Button>(R.id.btnProfile).setOnClickListener {
loadFragment(ProfileFragment())
}
findViewById<android.widget.Button>(R.id.btnSettings).setOnClickListener {
loadFragment(SettingsFragment())
}
}
private fun loadFragment(fragment: Fragment) {
supportFragmentManager.beginTransaction()
.replace(R.id.fragmentContainer, fragment)
.commit()
}
}
```

HomeFragment.kt

```
package com.example.fragmentnavigation
import android.os.Bundle
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import androidx.fragment.app.Fragment
class HomeFragment : Fragment() {
override fun onCreateView(
inflater: LayoutInflater,
container: ViewGroup?,
savedInstanceState: Bundle?
): View {
return inflater.inflate(
R.layout.fragment_home,
container,
false
)
}
}
```

ProfileFragment.kt

```
package com.example.fragmentnavigation
import android.os.Bundle
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import androidx.fragment.app.Fragment
class ProfileFragment : Fragment() {
override fun onCreateView(
inflater: LayoutInflater,
container: ViewGroup?,
savedInstanceState: Bundle?
): View {
return inflater.inflate(
R.layout.fragment_profile,
container,
false
)
}
}
```

SettingsFragment.kt

```
package com.example.fragmentnavigation
import android.os.Bundle
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import androidx.fragment.app.Fragment
class SettingsFragment : Fragment() {
override fun onCreateView(
inflater: LayoutInflater,
container: ViewGroup?,
savedInstanceState: Bundle?
): View {
return inflater.inflate(
R.layout.fragment_settings,
container,
false
)
}
}
```

fragment_home.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:gravity="center"
android:orientation="vertical">
<TextView
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="HOME SCREEN"
```

```
android:textSize="28sp"
android:textStyle="bold" />
<TextView
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:layout_marginTop="15dp"
android:text="Welcome to the Home Fragment"
android:textSize="18sp" />
</LinearLayout>
```

fragment_profile.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:gravity="center"
android:orientation="vertical">
<TextView
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="PROFILE SCREEN"
android:textSize="28sp"
android:textStyle="bold" />
<TextView
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:layout_marginTop="15dp"
android:text="This is the Profile Fragment"
android:textSize="18sp" />
</LinearLayout>
```

fragment_settings.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:gravity="center"
android:orientation="vertical">
<TextView
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="SETTINGS SCREEN"
android:textSize="28sp"
android:textStyle="bold" />
<TextView
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:layout_marginTop="15dp"
android:text="This is the Settings Fragment"
android:textSize="18sp" />
</LinearLayout>
```

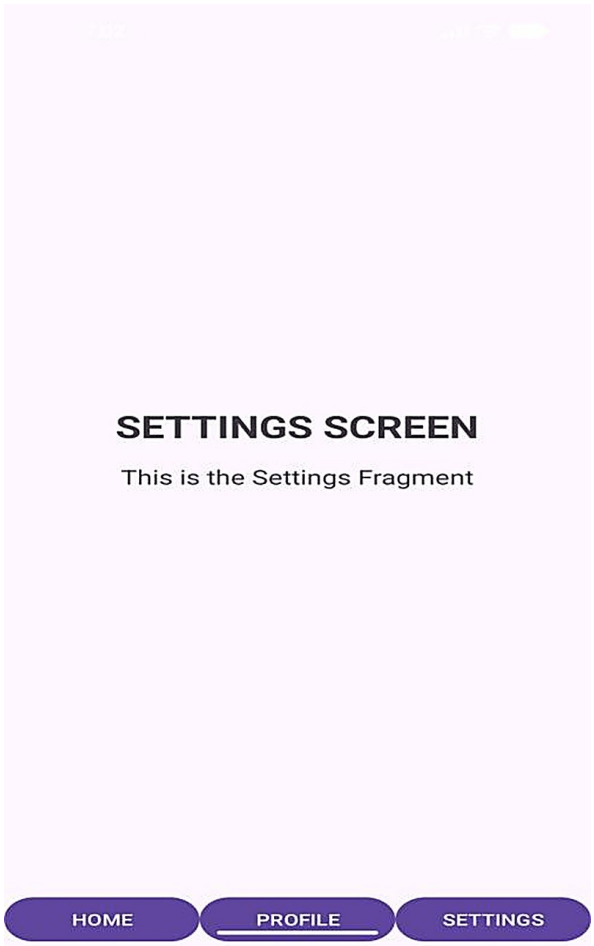
When the user clicks Home, Profile, or Settings, MainActivity replaces the current fragment. Clicking PROFILE changes the screen to the Profile screen, and clicking SETTINGS changes the

screen to the Settings screen.

Output:






SETTINGS SCREEN
This is the Settings Fragment
HOME
PROFILE
SETTINGS

PRACTICAL NO. 6

Develop an Android application demonstrating communication between Activity and Fragment.

MainActivity.kt

```
package com.example.fragmentcommunication
import android.os.Bundle
import android.widget.TextView
import androidx.appcompat.app.AppCompatActivity
class MainActivity : AppCompatActivity() {
lateinit var textView: TextView
override fun onCreate(savedInstanceState: Bundle?) {
super.onCreate(savedInstanceState)
setContentView(R.layout.activity_main)
textView = findViewById(R.id.txtMessage)
// Send data to Fragment
val fragment = FirstFragment()
val bundle = Bundle()
bundle.putString("message", "Hello from Activity")
fragment.arguments = bundle
supportFragmentManager.beginTransaction()
.replace(R.id.fragmentContainer, fragment)
.commit()
}
// Receive data from Fragment
fun receiveMessage(message: String) {
textView.text = message
}
}
```

FirstFragment.kt

```
package com.example.fragmentcommunication
import android.os.Bundle
import android.view.View
import android.widget.Button
import androidx.fragment.app.Fragment
class FirstFragment : Fragment(R.layout.fragment_first) {
override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
val btn = view.findViewById<Button>(R.id.btnSend)
// Get data from Activity
val message = arguments?.getString("message")
btn.text = message
// Send data to Activity
btn.setOnClickListener {
(activity as MainActivity).receiveMessage(
"Message received from Fragment"
)
}
}
}
```

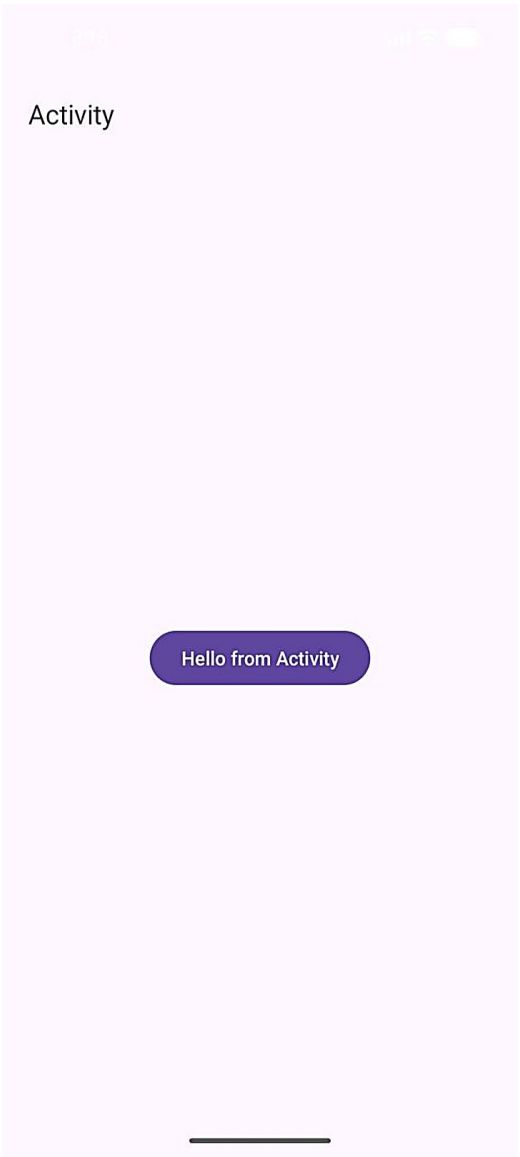
activity_main.xml

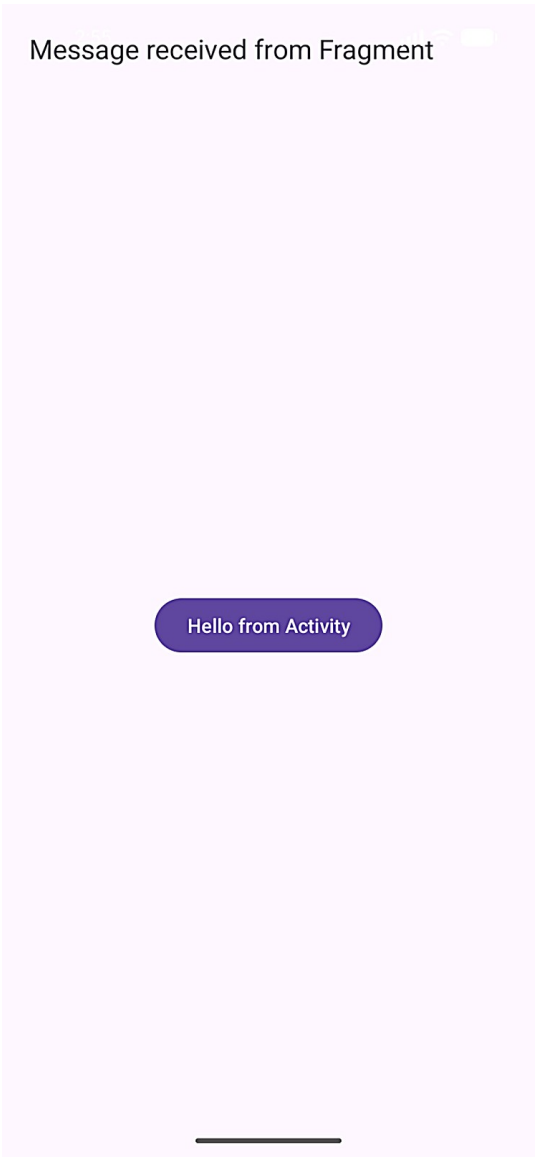
```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:orientation="vertical">
<TextView
android:id="@+id/txtMessage"
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:text="Activity"
android:layout_marginTop="48dp"
android:textSize="20sp"
android:padding="20dp"/>
<FrameLayout
android:id="@+id/fragmentContainer"
android:layout_width="match_parent"
android:layout_height="match_parent"/>
</LinearLayout>
```

fragment_first.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:gravity="center"
android:orientation="vertical">
<Button
android:id="@+id/btnSend"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="Send Message"/>
</LinearLayout>
```

Output :


Activity
Hello from Activity


Message received from Fragment
Hello from Activity

PRACTICAL NO. 7

Develop an Android application using LinearLayout (Horizontal and Vertical) to design structured UI.

This program demonstrates Vertical and Horizontal LinearLayout in Kotlin Android Studio.

activitymain.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:orientation="vertical"
android:padding="16dp">
<!-- Section 1: Horizontal Orientation -->
<TextView
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:text="Horizontal Orientation"
android:textAlignment="center"
android:textSize="18sp"
android:textStyle="bold"
android:textColor="#808080"
android:layout_marginBottom="12dp" />
<LinearLayout
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:orientation="horizontal"
android:layout_marginBottom="24dp">
<Button
android:layout_width="0dp"
android:layout_height="80dp"
android:layout_weight="1"
android:layout_marginEnd="4dp"
android:text="Button\n1"
android:textColor="#666666"
android:backgroundTint="#D3D3D3" />
<Button
android:layout_width="0dp"
android:layout_height="80dp"
android:layout_weight="1"
android:layout_marginStart="4dp"
android:layout_marginEnd="4dp"
android:text="Button\n1"
android:textColor="#666666"
android:backgroundTint="#D3D3D3" />
<Button
android:layout_width="0dp"
android:layout_height="80dp"
android:layout_weight="1"
android:layout_marginStart="4dp"
android:layout_marginEnd="4dp"
android:text="Button\n1"
```

```
android:textColor="#666666"
android:backgroundTint="#D3D3D3" />
<Button
android:layout_width="0dp"
android:layout_height="80dp"
android:layout_weight="1"
android:layout_marginStart="4dp"
android:text="Button\n1"
android:textColor="#666666"
android:backgroundTint="#D3D3D3" />
</LinearLayout>
<!-- Section 2: Vertical Orientation -->
<TextView
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:text="Vertical Orientation"
android:textAlignment="center"
android:textSize="18sp"
android:textStyle="bold"
android:textColor="#808080"
android:layout_marginBottom="12dp" />
<Button
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:text="Button2"
android:textColor="#666666"
android:backgroundTint="#D3D3D3"
android:layout_marginBottom="8dp" />
<Button
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:text="Button2"
android:textColor="#666666"
android:backgroundTint="#D3D3D3" />
</LinearLayout>
```

MainActivity.kt

No special Kotlin code is required:

```
package com.example.linearlayout
import android.os.Bundle
import androidx.appcompat.app.AppCompatActivity
class MainActivity : AppCompatActivity() {
override fun onCreate(savedInstanceState: Bundle?) {
super.onCreate(savedInstanceState)
setContentView(R.layout.activity_main)
}
}
```

Output:


8:00
LinearLayout
Horizontal Orientation
Button 1
Button 1
Button 1
Button 1
Vertical Orientation
Button2
Button2

PRACTICAL NO. 8

Develop an Android application using RelativeLayout to align UI components dynamically.

1. Activity_main.xml

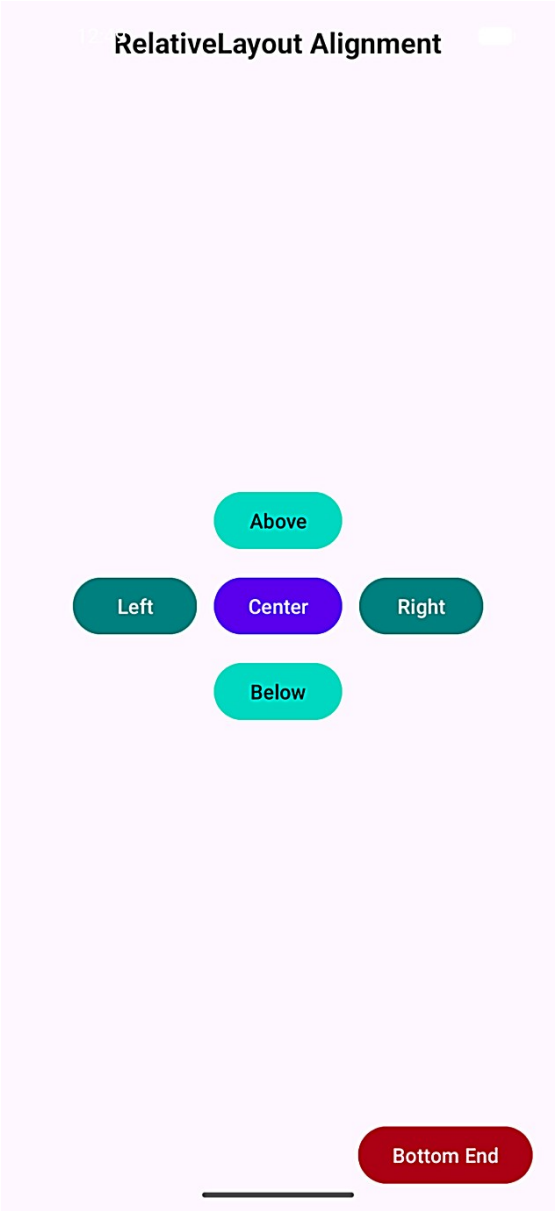
```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:padding="16dp">
<TextView
android:id="@+id/tvTitle"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:layout_alignParentTop="true"
android:layout_centerHorizontal="true"
android:text="RelativeLayout Alignment"
android:textColor="#333333"
android:textSize="20sp"
android:textStyle="bold" />
<Button
android:id="@+id/btnCenter"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:layout_centerInParent="true"
android:backgroundTint="#6200EE"
android:text="Center"
android:textColor="#FFFFFF" />
<Button
android:id="@+id/btnAbove"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:layout_above="@id/btnCenter"
android:layout_alignStart="@id/btnCenter"
android:layout_alignEnd="@id/btnCenter"
android:layout_marginBottom="12dp"
android:backgroundTint="#03DAC5"
android:text="Above"
android:textColor="#000000" />
<Button
android:id="@+id/btnBelow"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:layout_below="@id/btnCenter"
android:layout_alignStart="@id/btnCenter"
android:layout_alignEnd="@id/btnCenter"
android:layout_marginTop="12dp"
android:backgroundTint="#03DAC5"
android:text="Below"
android:textColor="#000000" />
<Button
android:id="@+id/btnLeft"
```

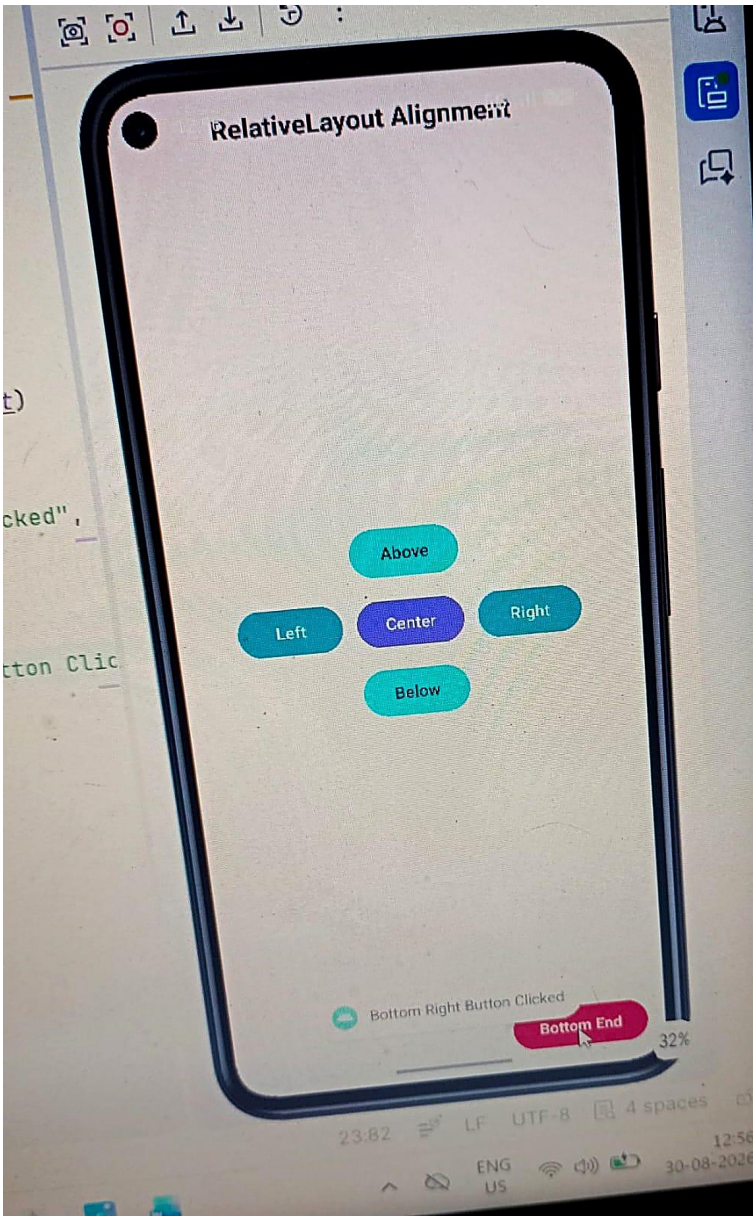
```
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:layout_alignTop="@id/btnCenter"
android:layout_marginEnd="12dp"
android:layout_toStartOf="@id/btnCenter"
android:backgroundTint="#018786"
android:text="Left"
android:textColor="#FFFFFF" />
<Button
android:id="@+id/btnRight"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:layout_alignTop="@id/btnCenter"
android:layout_marginStart="12dp"
android:layout_toEndOf="@id/btnCenter"
android:backgroundTint="#018786"
android:text="Right"
android:textColor="#FFFFFF" />
<Button
android:id="@+id/btnBottomRight"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:layout_alignParentBottom="true"
android:layout_alignParentEnd="true"
android:backgroundTint="#B00020"
android:text="Bottom End"
android:textColor="#FFFFFF" />
</RelativeLayout>
```

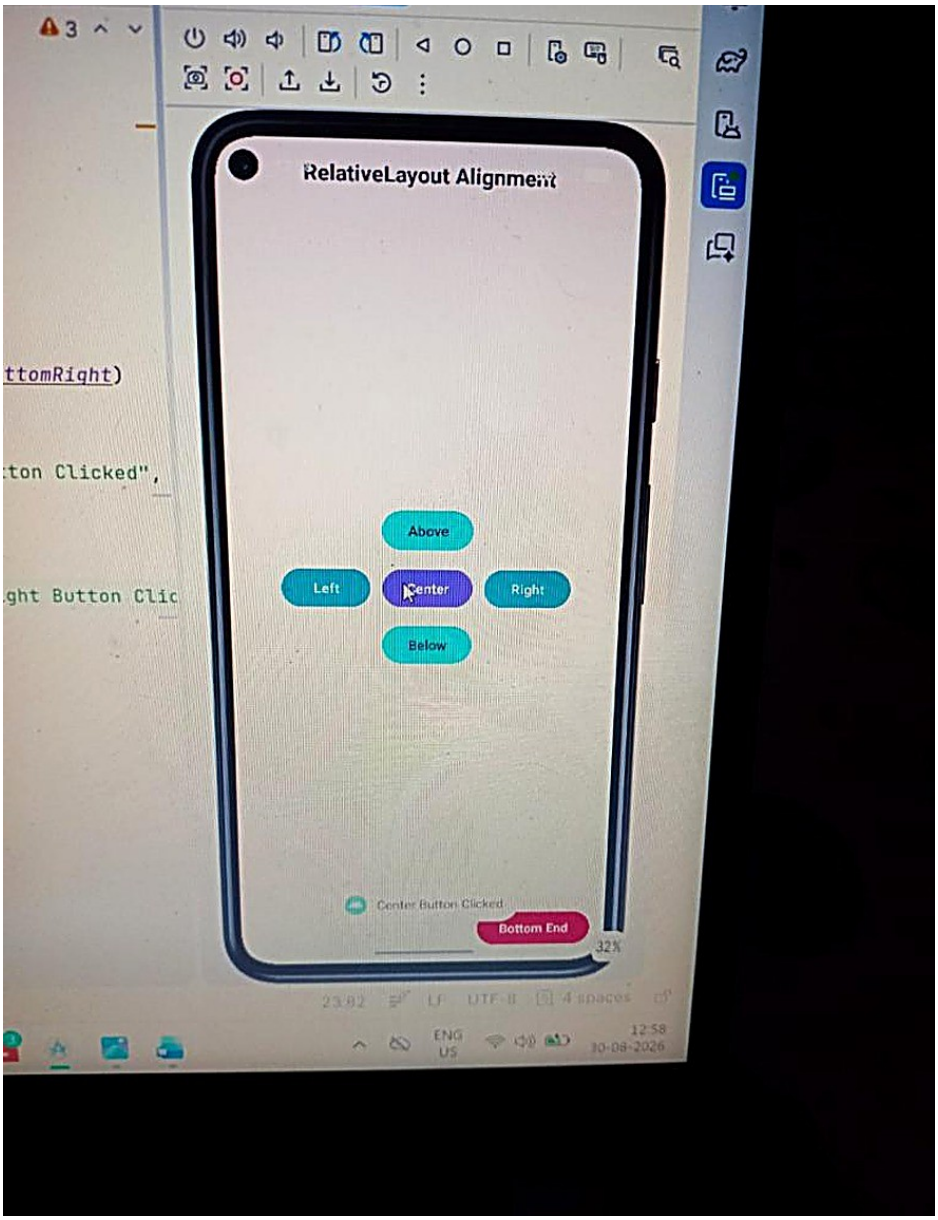
MainActivity.kt

```
package com.example.relativelayoutapp
import android.os.Bundle
import android.widget.Button
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity
class MainActivity : AppCompatActivity() {
override fun onCreate(savedInstanceState: Bundle?) {
super.onCreate(savedInstanceState)
setContentView(R.layout.activity_main)
val btnCenter = findViewById<Button>(R.id.btnCenter)
val btnBottomRight = findViewById<Button>(R.id.btnBottomRight)
btnCenter.setOnClickListener {
Toast.makeText(this, "Center Button Clicked", Toast.LENGTH_SHORT).show()
}
btnBottomRight.setOnClickListener {
Toast.makeText(this, "Bottom Right Button Clicked", Toast.LENGTH_SHORT).show()
}
}
}
```

Output:


RelativeLayout Alignment
Above
Left
Center
Right
Below
Bottom End


RelativeLayout Alignment
Above
Left
Center
Right
Below
Bottom Right Button Clicked
Bottom End
32%


RelativeLayout Alignment
Above
Left
Center
Right
Below
Center Button Clicked
Bottom End

PRACTICAL NO. 9

Develop an Android application using TableLayout to design a structured Login/Registration Form.

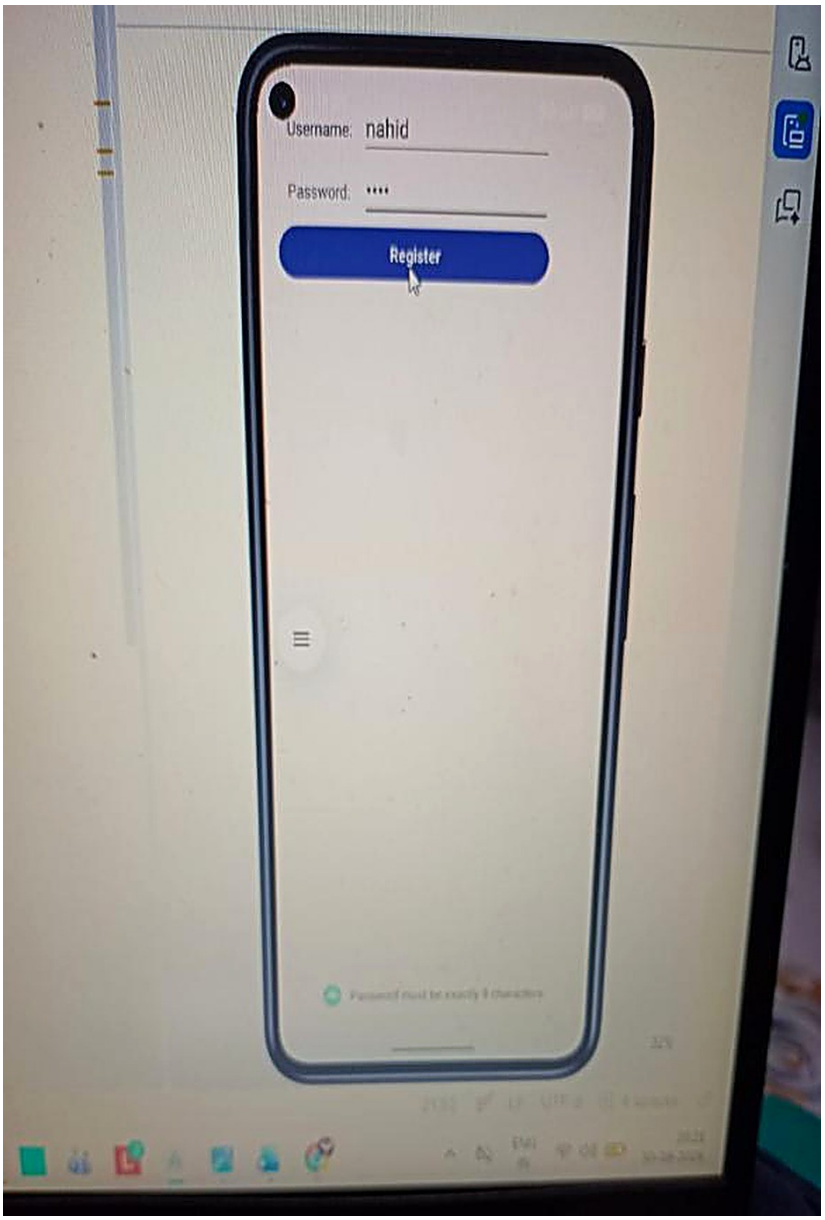
activity_main.xml

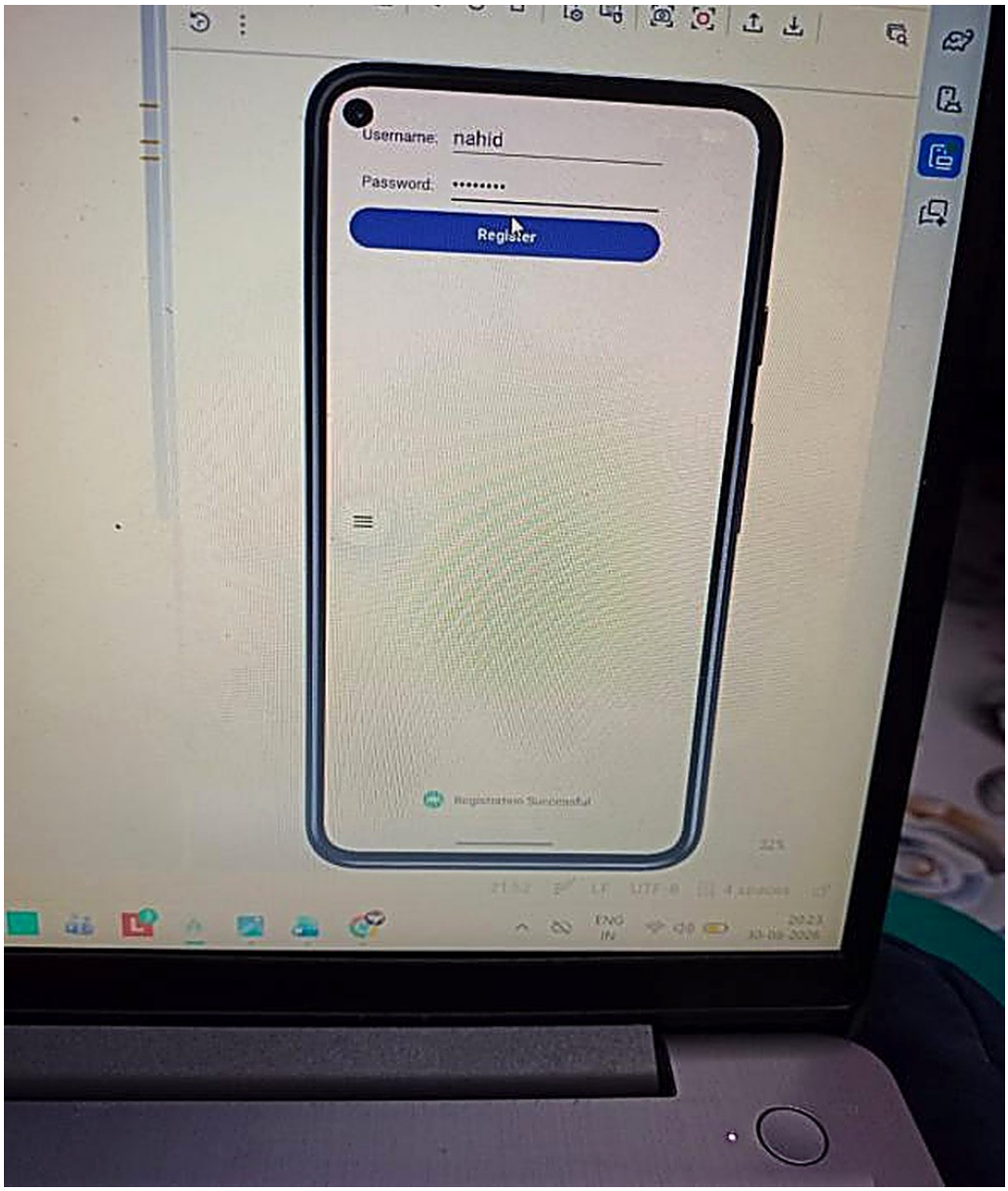
```
<?xml version="1.0" encoding="utf-8"?>
<TableLayout xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:padding="20dp">
<TableRow>
<TextView
android:text="Username:"
android:padding="10dp" />
<EditText
android:hint="Enter Username"
android:layout_width="200dp" />
</TableRow>
<TableRow>
<TextView
android:text="Password:"
android:padding="10dp" />
<EditText
android:hint="Enter Password"
android:inputType="textPassword"
android:layout_width="200dp" />
</TableRow>
<TableRow>
<Button
android:id="@+id/btnRegister"
android:text="Register"
android:layout_span="2" />
</TableRow>
</TableLayout>
```

MainActivity.kt

```
package com.example.tablelayout
import android.os.Bundle
import androidx.appcompat.app.AppCompatActivity
class MainActivity : AppCompatActivity() {
override fun onCreate(savedInstanceState: Bundle?) {
super.onCreate(savedInstanceState)
setContentView(R.layout.activity_main)
val btnRegister = findViewById<Button>(R.id.btnRegister)
btnRegister.setOnClickListener {
Toast.makeText(this, "Registration Successful", Toast.LENGTH_SHORT).show()
}
}
}
```

Output:


Username: nahid
Password: ••••
Register


Username: nahid
Password: ••••••••
Register
Registration Successful

PRACTICAL NO. 10

Develop an Android application using ScrollView to display long scrollable content.

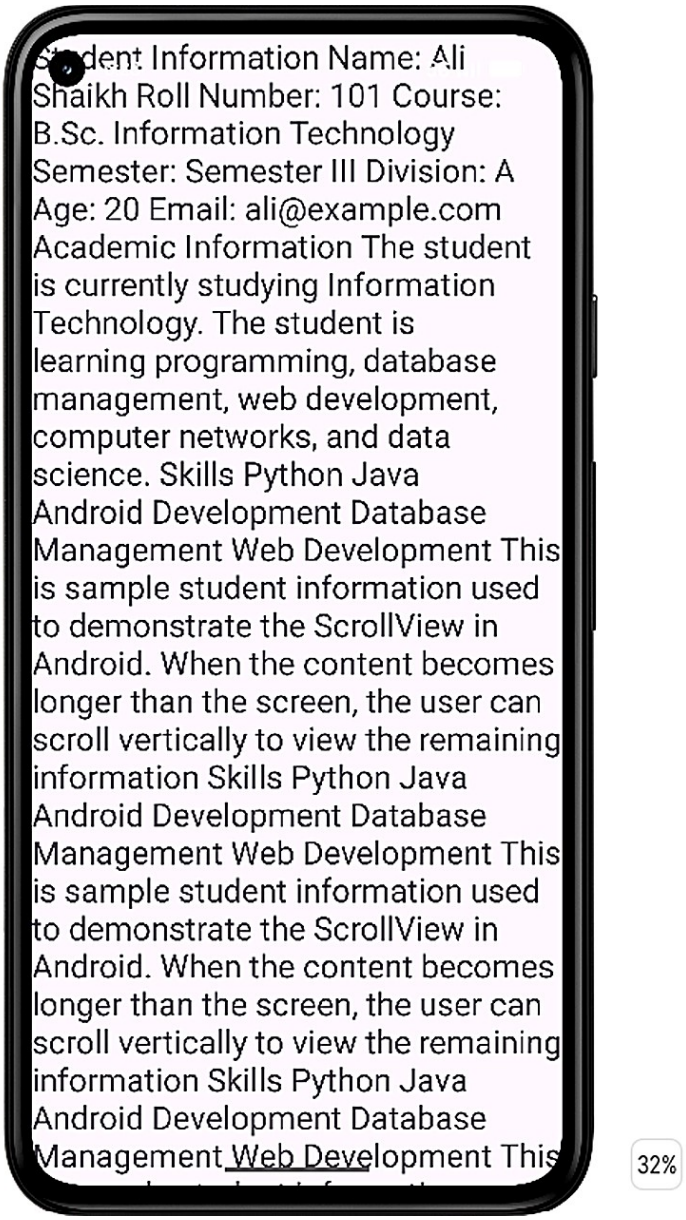
activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="match_parent"
android:layout_height="match_parent">
<LinearLayout
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:orientation="vertical"
android:padding="20dp">
<TextView
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:text="Student Information"
android:textSize="25sp" />
<TextView
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:text="@string/sample_text" />
</LinearLayout>
</ScrollView>
```

MainActivity.kt

```
package com.example.scrollview
import android.os.Bundle
import androidx.appcompat.app.AppCompatActivity
class MainActivity : AppCompatActivity() {
override fun onCreate(savedInstanceState: Bundle?) {
super.onCreate(savedInstanceState)
setContentView(R.layout.activity_main)
}
}
```

Output:


Student Information Name: Ali Shaikh Roll Number: 101 Course: B.Sc. Information Technology Semester: Semester III Division: A Age: 20 Email: ali@example.com Academic Information The student is currently studying Information Technology. The student is learning programming, database management, web development, computer networks, and data science. Skills Python Java Android Development Database Management Web Development This is sample student information used to demonstrate the ScrollView in Android. When the content becomes longer than the screen, the user can scroll vertically to view the remaining information Skills Python Java Android Development Database Management Web Development This is sample student information used to demonstrate the ScrollView in Android. When the content becomes longer than the screen, the user can scroll vertically to view the remaining information Skills Python Java Android Development Database Management Web Development This
32%

PRACTICAL NO. 11

Develop an Android application using Absolute Layout demonstrating fixed positioning.

Activity_main.xml

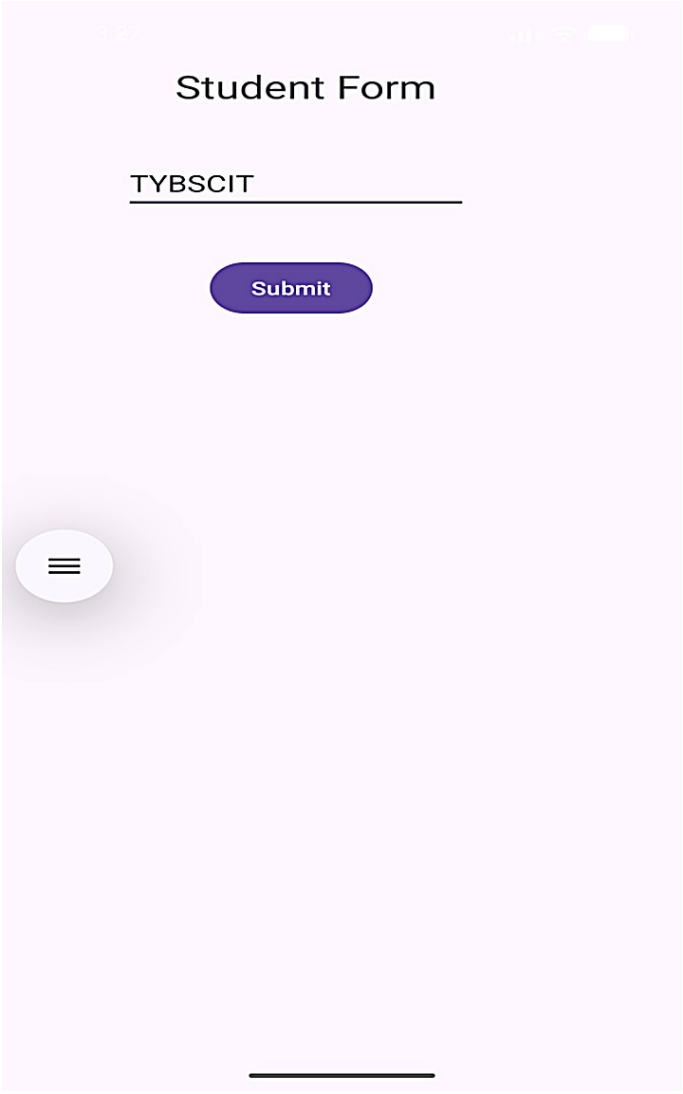
```
<?xml version="1.0" encoding="utf-8"?>
<AbsoluteLayout
xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="match_parent"
android:layout_height="match_parent">
<TextView
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="Student Form"
android:textSize="25sp"
android:layout_x="100dp"
android:layout_y="50dp" />
<EditText
android:id="@+id/edtName"
android:layout_width="200dp"
android:layout_height="wrap_content"
android:hint="Enter Name"
android:layout_x="70dp"
android:layout_y="120dp" />
<Button
android:id="@+id/btnSubmit"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="Submit"
android:layout_x="120dp"
android:layout_y="200dp" />
</AbsoluteLayout>
```

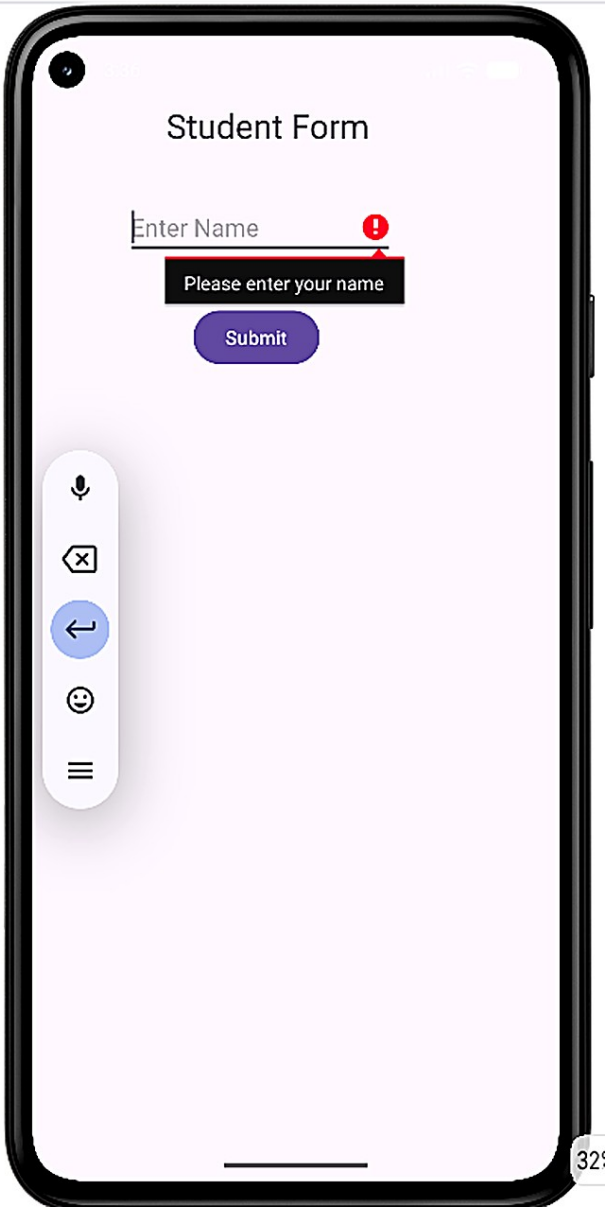
MainActivity.kt

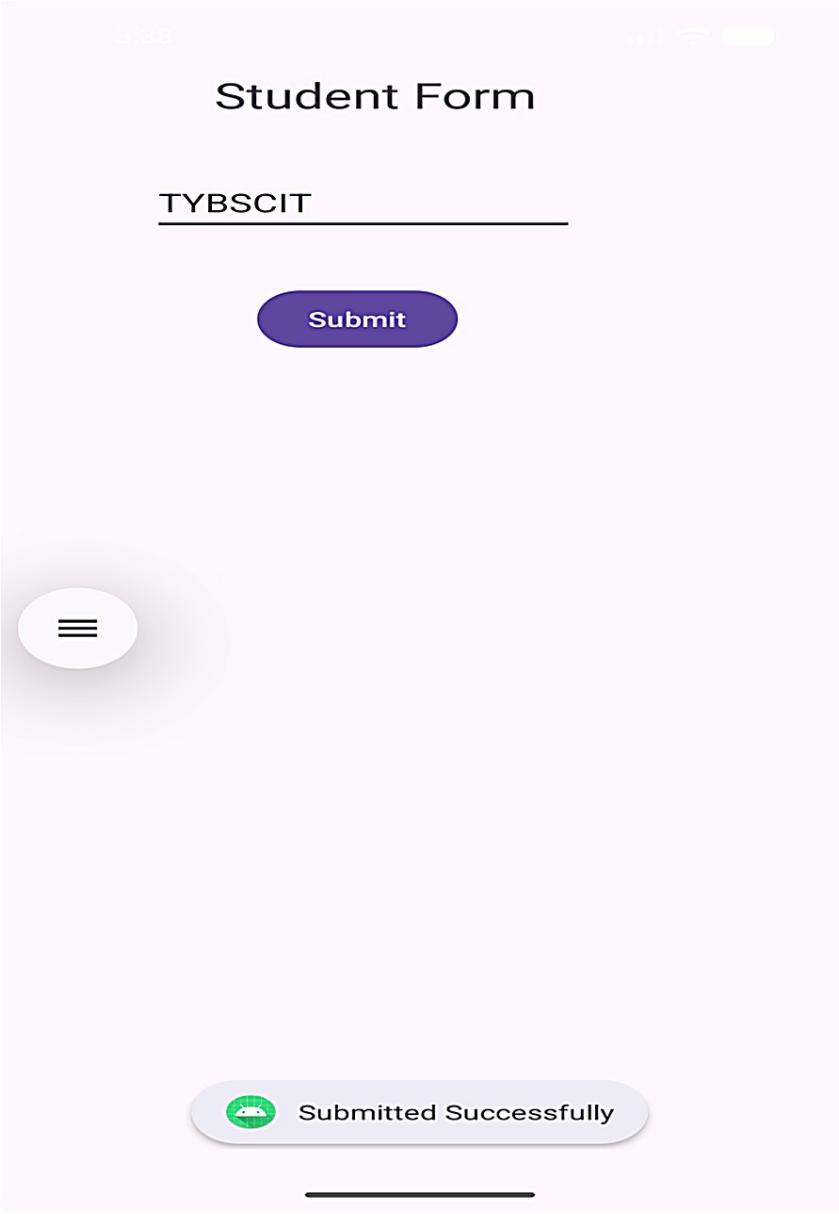
```
package com.example.relativelayout
import android.os.Bundle
import android.widget.Button
import android.widget.EditText
import android.widget.TextView
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity
class MainActivity : AppCompatActivity() {
override fun onCreate(savedInstanceState: Bundle?) {
super.onCreate(savedInstanceState)
setContentView(R.layout.activity_main)
val name = findViewById<EditText>(R.id.edtName)
val button = findViewById<Button>(R.id.btnSubmit)
button.setOnClickListener {
if (name.text.toString().trim().isEmpty()) {
name.error = "Please enter your name"
name.requestFocus()
} else {
```

```
Toast.makeText(
this,
"Submitted Successfully",
Toast.LENGTH_SHORT
).show()
}
}
}
}
```

Output:




Student Form
Enter Name
Please enter your name
Submit


Student Form
TYBSCIT
Submit
Submitted Successfully

PRACTICAL NO. 12

Develop an Android application using FrameLayout to overlay UI elements.

FrameLayout is used to place or overlay UI elements on top of each other.

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<FrameLayout
xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="match_parent"
android:layout_height="match_parent">
<ImageView
android:layout_width="match_parent"
android:layout_height="match_parent"
android:src="@drawable/img"
android:scaleType="centerCrop" />
<TextView
android:id="@+id/textView"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:layout_gravity="center"
android:text="Beautiful Nature"
android:textColor="#FFFFFF"
android:textSize="30sp"
android:textStyle="bold"
android:background="#80000000"
android:padding="20dp" />
<Button
android:id="@+id/button"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:layout_gravity="bottom|center_horizontal"
android:layout_marginBottom="50dp"
android:text="Change Text" />
</FrameLayout>
```

MainActivity.kt

```
package com.example.relativelayout
import android.os.Bundle
import android.widget.Button
import android.widget.TextView
import androidx.appcompat.app.AppCompatActivity
class MainActivity : AppCompatActivity() {
override fun onCreate(savedInstanceState: Bundle?) {
super.onCreate(savedInstanceState)
setContentView(R.layout.activity_main)
val textView = findViewById<TextView>(R.id.textView)
val button = findViewById<Button>(R.id.button)
button.setOnClickListener {
textView.text = "Welcome to Android"
}
}
```

}

Output :


12:47
5G
Beautiful Nature
Change Text


12:48
5G
Welcome to Android
Change Text

PRACTICAL NO. 13

Develop an Android application using ListView with ArrayAdapter and implement item click handling.

This program displays a list of students and shows a message when an item is clicked.

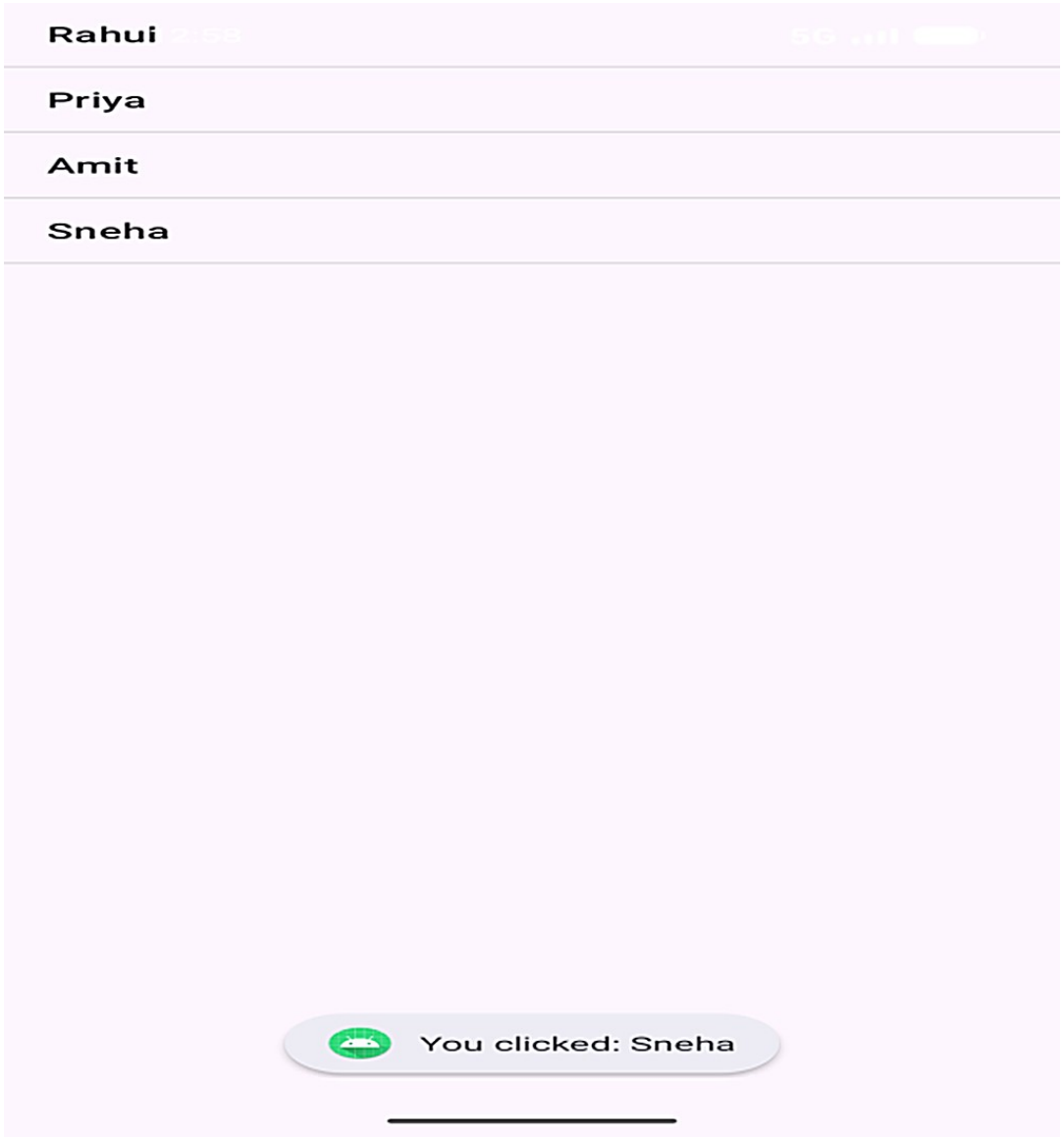
1. activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<ListView xmlns:android="http://schemas.android.com/apk/res/android"
android:id="@+id/listView"
android:layout_width="match_parent"
android:layout_height="match_parent" />
```

2. MainActivity.kt

```
package com.example.listview
import android.os.Bundle
import android.widget.ArrayAdapter
import android.widget.ListView
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity
class MainActivity : AppCompatActivity() {
override fun onCreate(savedInstanceState: Bundle?) {
super.onCreate(savedInstanceState)
setContentView(R.layout.activity_main)
val listView = findViewById<ListView>(R.id.listView)
val students = arrayOf(
"Rahul",
"Priya",
"Amit",
"Sneha"
)
val adapter = ArrayAdapter(
this,
android.R.layout.simple_list_item_1,
students
)
listView.adapter = adapter
listView.setOnItemClickListener { _, _, position, _ ->
Toast.makeText(
this,
"You clicked: ${students[position]}",
Toast.LENGTH_SHORT
).show()
}
}
}
```

Output :


Rahui
Priya
Amit
Sneha
You clicked: Sneha


Rahul
Priya
Amit
Sneha
You clicked: Priya

PRACTICAL NO. 14

Develop an Android application using GridView with Custom Adapter and image selection handling.

1. Create the GridView Layout

Create/replace activity_main.xml:

```
<?xml version="1.0" encoding="utf-8"?>
<GridView
xmlns:android="http://schemas.android.com/apk/res/android"
android:id="@+id/gridView"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:numColumns="2"
android:verticalSpacing="15dp"
android:horizontalSpacing="15dp"
android:padding="15dp"
android:gravity="center"
android:stretchMode="columnWidth"/>
```

2. Create the Grid Item Layout

Create a new XML file: grid_item.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:orientation="vertical"
android:gravity="center"
android:padding="8dp">
<ImageView
android:id="@+id/imageView"
android:layout_width="120dp"
android:layout_height="120dp"
android:scaleType="centerCrop" />
<TextView
android:id="@+id/textView"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="Image Name"
android:textSize="18sp"
android:textStyle="bold"
android:padding="8dp" />
</LinearLayout>
```

3. Add Images

Put your images inside: app -> res -> drawable

apple.png, banana.png, mango.png, orange.png, grapes.png

MainActivity.kt

```
package com.example.relativelayout
import android.os.Bundle
```

```
import android.widget.ArrayAdapter
import android.widget.Button
import android.widget.GridView
import android.widget.ListView
import android.widget.TextView
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity
class MainActivity : AppCompatActivity() {
override fun onCreate(savedInstanceState: Bundle?) {
super.onCreate(savedInstanceState)
setContentView(R.layout.activity_main)
val gridView = findViewById<GridView>(R.id.gridView)
// Image names
val images = arrayOf(
R.drawable.img,
R.drawable.img_1,
R.drawable.img_2,
R.drawable.img_3,
R.drawable.img_5
)
// Image names
val names = arrayOf(
"Scenery",
"MOON",
"SunRise",
"Tree",
"Mountain"
)
// Set adapter to GridView
gridView.adapter = adapter
// Image selection handling
gridView.setOnItemClickListener { parent, view, position, id ->
Toast.makeText(
this,
"You selected: ${names[position]}",
Toast.LENGTH_SHORT
).show()
}
}
}
```

Create the Custom Adapter

Create a Kotlin class: ImageAdapter.kt

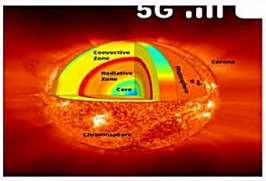
```
package com.example.relativelayout
import android.content.Context
import android.view.*
import android.widget.*
class ImageAdapter(
private val context: Context,
private val images: Array<Int>,
private val names: Array<String>
) : BaseAdapter() {
override fun getCount(): Int {
```

```
return images.size
}
override fun getItem(position: Int): Any {
return images[position]
}
override fun getItemId(position: Int): Long {
return position.toLong()
}
override fun getView(
position: Int,
convertView: View?,
parent: ViewGroup?
): View {
val view = LayoutInflater.from(context)
.inflate(R.layout.grid_item, parent, false)
val imageView = view.findViewById<ImageView>(R.id.imageView)
val textView = view.findViewById<TextView>(R.id.textView)
imageView.setImageResource(images[position])
textView.text = names[position]
return view
}
}
```

Output :



Scenery



MOON



SunRise



Tree



Mountain

You selected: Mountain

PRACTICAL NO. 15

Develop an Android application implementing RecyclerView as a modern alternative to ListView.

1. activity_main.xml

```
<androidx.recyclerview.widget.RecyclerView
xmlns:android="http://schemas.android.com/apk/res/android"
android:id="@+id/recyclerView"
android:layout_width="match_parent"
android:layout_height="match_parent"/>
```

2. item.xml

```
<TextView
xmlns:android="http://schemas.android.com/apk/res/android"
android:id="@+id/textView"
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:padding="20dp"
android:textSize="20sp"/>
```

3. MainActivity.kt

```
package com.example.recyclerview
import android.os.Bundle
import androidx.appcompat.app.AppCompatActivity
import androidx.recyclerview.widget.LinearLayoutManager
import androidx.recyclerview.widget.RecyclerView
class MainActivity : AppCompatActivity() {
override fun onCreate(savedInstanceState: Bundle?) {
super.onCreate(savedInstanceState)
setContentView(R.layout.activity_main)
val list = arrayOf("Aisha", "Rahul", "Fatima", "Aman")
val rv = findViewById<RecyclerView>(R.id.recyclerView)
rv.layoutManager = LinearLayoutManager(this)
rv.adapter = object : RecyclerView.Adapter<VH>() {
override fun onCreateViewHolder(p: ViewGroup, v: Int) =
VH(layoutInflater.inflate(R.layout.item, p, false))
override fun onBindViewHolder(h: VH, p: Int) {
h.textView.text = list[p]
}
override fun getItemCount() = list.size
}
}
}
class VH(v: View) : RecyclerView.ViewHolder(v) {
val textView = v.findViewById<TextView>(R.id.textView)
}
```

Output :

Displays a simple student list using RecyclerView.

Aisha

Rahul

Fatima

Aman